

Projeto e Análise de Algoritmos

Lista 1

FGV - EMap

2026

Para os próximos exercícios você deve alterar o arquivo `lista-1-paa.py` adicionando a solução no corpo da função correspondente.

Atenção:

- Não altere o nome do arquivo Python.
- Para cada questão abaixo existe uma função definida no arquivo Python.
- Não altere a assinatura das funções. Você deve receber e retornar os tipos definidos.
- Caso queira definir uma classe ou função auxiliar, faça isso dentro da função do problema que estiver resolvendo.

1 Recorrência com Busca Embutida

Determine uma função $f(n)$ tal que $T(n) = \Theta(f(n))$ para o código a seguir. Explique como a solução foi encontrada, apresentando a equação de recorrência e o método usado para resolvê-la.

```
int solve(int v[], int left, int right) {
    int n = right - left + 1;

    if (n <= 1)
        return 0;

    int mid = (left + right) / 2;

    int count = solve(v, left, mid)
                + solve(v, mid + 1, right);

    for (int i = left; i <= mid; i++) {
        // Conta quantos elementos de v[mid+1..right] são menores ou iguais a v[i].
        count += binarySearchCount(v, mid + 1, right, v[i]);
    }

    merge(v, left, mid, right);

    return count;
}
```

Considere que `binarySearchCount` executa uma busca binária padrão ($O(\log n)$) e que `merge` executa uma mesclagem padrão de duas sequências ordenadas ($O(n)$).

2 Painel de Sensores

Uma plantação de precisão organiza seus sensores de temperatura em uma grade $n \times n$ de leituras inteiras, em que cada linha está ordenada de forma crescente da esquerda para a direita, e cada coluna está ordenada de forma crescente de cima para baixo. Isso não significa que a grade inteira, lida em sequência, esteja ordenada.

A central de controle quer saber, periodicamente, qual é a k -ésima menor leitura entre todos os n^2 sensores. Sua solução deve custar $O(n \log D)$, onde D é a diferença entre a maior e a menor leitura da grade.

Restrições:

- $1 \leq n \leq 1000$
- $1 \leq k \leq n^2$

Exemplos:

Entrada	Saída
grade = [(2, 6, 9), (8, 9, 12), (11, 13, 14)] e $k = 5$	9
grade = [(2, 6, 9), (8, 9, 12), (11, 13, 14)] e $k = 1$	2
grade = [(-15, -10, -5), (-12, -8, -2), (-9, -4, 0)] e $k = 4$	-9
grade = [(-10, -5, 2), (-4, 0, 6), (1, 5, 10)] e $k = 7$	5

3 Maratona de Podcasts

Você tem uma lista com n episódios de podcast, na ordem em que foram publicados, cada um pertencente a uma categoria (identificada por um inteiro qualquer, que não segue nenhum padrão ou intervalo conhecido). Você quer maratonar o maior trecho contínuo de episódios possível, mas para manter a vibe, o trecho escolhido não pode conter mais do que k categorias diferentes.

Determine o tamanho do maior trecho contínuo de episódios que respeita esse limite de k categorias distintas em tempo linear.

Restrições:

- $1 \leq k \leq n \leq 10^6$
- Os identificadores de categoria são inteiros arbitrários, até 10^9 em valor absoluto.

Exemplos:

Entrada	Saída
categorias = [1, 2, 1, 2, 3] e $k = 2$	4
categorias = [1, 2, 3, 4] e $k = 1$	1
categorias = [4, 4, 4, 4] e $k = 1$	4

4 Central de Senhas

Um sistema de segurança só permite alterar uma senha trocando exatamente um caractere por vez, e cada senha intermediária do processo precisa já constar em um banco de senhas cadastradas como seguras (uma lista de tamanho N de strings, todas com o mesmo comprimento L ,

compostas apenas dígitos de '0' a '9'). Dada uma senha inicial e uma senha final (ambas garantidamente presentes no banco), determine o menor número de trocas necessárias para transformar uma na outra usando apenas senhas do banco como passos intermediários em complexidade de tempo $O(N \cdot L)$. Se não for possível, retorne -1.

Restrições:

- $1 \leq L \leq 20$
- $1 \leq N \leq 5000$

Exemplos:

Entrada	Saída
inicial = "1000", final = "1003", banco = ["1000", "1003"]	1
inicial = "1000", final = "1103";, banco = ["1000", "1003", "1103"]	2
inicial = "1000", final = "2200";, banco = ["1000", "1100", "2200"]	-1

5 Cache Inteligente

Um navegador quer manter um cache de capacidade fixa C com o conteúdo das páginas carregadas mais recentemente, para evitar recarregá-las. O cache deve suportar duas operações, ambas com complexidade $O(1)$ em média.

As duas operações são: `consultar(chave)`, que retorna o valor associado à chave e marca essa chave como a mais recentemente usada caso essa exista no cache e caso não exista, retorna -1; e `atualizar(chave, valor)`, que insere ou atualiza o valor associado à chave, marcando-a como a mais recentemente usada. Se o cache já estiver cheio e a chave for nova, remove antes a entrada menos recentemente usada.

Projete a estrutura de dados e implemente as duas operações.

Restrições:

- $1 \leq C \leq 10^5$ (capacidade do cache)
- Até 10^6 chamadas de `consultar/atualizar` no total

Exemplo:

Entrada	Saída
cache = Cache(capacidade=2)	-
cache.atualizar("home", "conteudo_home")	-
cache.atualizar("sobre", "conteudo_sobre")	-
cache.consultar("home")	"conteudo_home"
cache.atualizar("contato", "conteudo_contato")	remove a chave "sobre"
cache.consultar("sobre")	-1
cache.consultar("contato")	conteudo_contato

6 Rádio Digital

Uma plataforma de rádio digital quer exibir, ao final do dia, as k músicas mais tocadas, em ordem decrescente de número de reproduções. A plataforma possui um histórico com n reproduções (identificadores de música podem se repetir livremente) e sabe-se que o número de reproduções de qualquer música está sempre entre 0 e n .

Projete um algoritmo que produza o ranking das k músicas mais tocadas com complexidade $O(n)$ no total. Caso o número de músicas tocadas seja menor que k , retorna-se todas.

Restrições:

- $1 \leq n \leq 10^6$
- $1 \leq k \leq n$
- Cada identificador de música é um inteiro entre 0 e n

Exemplos:

Entrada	Saída
reproducoes = [1, 1, 1, 2, 2, 3] e $k=2$	[1, 2]
reproducoes = [1, 1, 1, 1, 1, 1] e $k=2$	[1]

7 Análise de Crédito

Uma financeira analisa o score de crédito (um inteiro) de milhões de solicitantes, armazenados em uma lista não ordenada. O time de risco quer saber, com frequência, qual é o k -ésimo menor score entre todos os solicitantes, sem ordenar a lista inteira a cada consulta.

- Projete um algoritmo com tempo esperado $O(n)$ (caso médio) para resolver o problema.
- A financeira vai usar esse algoritmo em decisões automatizadas e não pode aceitar que, em alguma entrada especialmente desfavorável (mesmo que rara), o tempo de execução degrade para $O(n^2)$. Explique por que o algoritmo do item (a) pode degradar nesse cenário e adapte-o para garantir $O(n)$ também no pior caso.

Restrições:

- $1 \leq n \leq 10^6$
- $1 \leq k \leq n$

Exemplos:

Entrada	Saída
$A = [70, 10, 40, 3, 20, 15]$ e $k = 3$	15
$A = [70, 10, 40, 3, 20, 15]$ e $k = 1$	3

8 Feed Consolidado

Um usuário de rede social segue k comunidades; cada comunidade produz uma lista de posts já ordenada por horário de publicação (um vetor com k listas, cada uma ordenada de forma crescente pelo timestamp). Ao abrir o feed, o aplicativo precisa exibir todos os posts de todas as comunidades seguidas em um único feed consolidado, também ordenado por horário.

Sabendo que o número total de posts somando todas as k comunidades é n , projete um algoritmo que produza o feed consolidado com complexidade $O(n \log k)$. Explique por que mesclar as listas duas a duas, sequencialmente, não atinge esse limite, e qual estrutura permite alcançá-lo.

Restrições:

- $1 \leq k \leq n \leq 10^6$ (k = número de comunidades, n = total de posts)
- Os timestamps são inteiros, com $1 \leq \text{timestamp} \leq 10^9$

Exemplo:

Entrada	Saída
comunidades = $[[1, 4, 5], [1, 3, 4], [2, 6]]$	$[1, 1, 2, 3, 4, 4, 5, 6]$

9 Autonomia Elétrica

Uma frota de carros elétricos precisa percorrer uma rota circular com n estações de recarga, numeradas de 0 a $n - 1$. Na estação i , o carro recarrega energia[i] unidades e gasta exatamente consumo[i] unidades para chegar à próxima estação (a estação 0 é a sucessora da estação $n-1$). A bateria começa vazia e não há limite de capacidade de carga.

- Descreva uma solução de força bruta para determinar se existe uma estação de partida a partir da qual é possível completar o circuito inteiro sem a bateria ficar negativa em algum trecho, retornando o índice de tal estação ou -1 caso não exista, e qual sua complexidade.
- A fabricante garante que, se existir uma solução, ela é única. Usando esse fato, encontre uma forma de resolver o mesmo problema com complexidade $O(n)$, sem testar múltiplos pontos de partida isoladamente. Justifique por que sua estratégia sempre encontra a estação correta, caso ela exista.

Restrições:

- $1 \leq n \leq 10^5$
- $0 \leq \text{energia}[i], \text{consumo}[i] \leq 10^9$ para todo $0 \leq i < n$

Exemplos:

Entrada	Saída
energia = $[3, 1, 1]$ e consumo = $[1, 2, 2]$	0
energia = $[2, 3, 5]$ e consumo = $[3, 5, 4]$	-1

10 Trilha de Picos de Montanha

Um alpinista tem um mapa com as altitudes de n pontos de checagem ao longo de uma trilha, na ordem em que aparecem no percurso. Ele quer escolher um subconjunto desses pontos — mantendo a ordem original em que aparecem na trilha — de forma que a altitude seja estritamente crescente a cada ponto escolhido, formando a trilha ascendente mais longa possível.

Determine o tamanho dessa trilha mais longa. Sua solução deve ter complexidade de tempo $O(n \cdot \log n)$.

Restrições:

- $1 \leq n \leq 5 \cdot 10^5$
- As altitudes são inteiras e podem se repetir na lista original (mas não na trilha escolhida, que deve ser estritamente crescente).

Exemplos:

Entrada	Saída
altitudes = [3, 1, 4, 1, 5, 9, 2, 6]	4
altitudes = [5, 4, 3, 2, 1]	1